

Learning to Play Wordle: Hierarchical Reinforcement Learning with Linguistic Priors

Yutong He

CL, Heidelberg University.

Keywords: Reinforcement learning, hierarchical representations, interpretability, Wordle.

Abstract. We use Wordle to investigate how pretrained linguistic action representations and hierarchical policy structure affect reinforcement learning. We compare flat and hierarchical PPO policies using either pretrained language-model or random word embeddings. Pretrained representations substantially improve learning, while hierarchical structure has a smaller effect on performance. Linear probing shows that task information is decodable early in training and that position-specific information is strongest in low-level hierarchical representations.

1 Introduction

Wordle¹ is a word-guessing game in which a player has six attempts to identify a hidden five-letter word. After each guess, each letter receives a color-coded signal: gray indicates that the letter is absent from the target word, yellow that it is present but in a different position, and green that it is in the correct position. Despite its simple rules, solving Wordle requires a sequence of decisions that balance information gathering with exploiting knowledge acquired from previous guesses. This makes Wordle a compact environment for studying sequential decision making and reinforcement learning (RL).

A number of computational approaches to Wordle have focused primarily on solving the game efficiently. Information-theoretic solvers select guesses that maximize the expected reduction in uncertainty over possible solutions, while exact dynamic-programming approaches can compute an optimal policy under a fixed vocabulary [1]. Wordle has also been formulated as a partially observable decision problem and addressed using reinforcement learning and rollout methods [2]. These approaches demonstrate that Wordle can be treated as a sequential decision problem, while its large discrete vocabulary presents an interesting challenge for RL. In this project, however, our main interest is not to construct an optimal Wordle solver, but to use Wordle as a small and interpretable environment for studying the strategies and representations learned by RL agents.

Human Wordle strategies often involve recognizable intermediate goals. For example, early guesses may be chosen to identify common vowels or letters, while later guesses increasingly exploit known letters and positions. This motivates our first question: whether similarly interpretable strategic structure can emerge in a learned policy without explicitly specifying such strategies.

Hierarchical reinforcement learning (HRL) provides a useful motivation for organizing a policy at multiple levels. [3] describe HRL as a computational framework in which complex behavior can be organized through higher-level representations and reusable subroutines. Inspired by this perspective, we introduce a simplified hierarchical policy architecture for Wordle. Rather than implementing discrete options or temporally extended actions, the hierarchical agent first maps the current game state to a continuous latent strategy representation. This representation is

¹You can play the official Wordle at <https://www.nytimes.com/games/wordle/index.html>.

then combined with the game state by a lower-level network that selects the next word. The intermediate strategy representation therefore provides an explicit representational hierarchy between the observed state and the word-level policy. We investigate whether this latent representation develops interpretable structure related to different aspects of Wordle solving.

A second question concerns the role of prior linguistic knowledge. Words are not arbitrary discrete actions, but contain systematic orthographic and lexical structure. We therefore use a pretrained character-level language model (LM) to provide representations of candidate word actions. The LM does not generate guesses or determine which words are legal; instead, each legal word is mapped to a fixed embedding. This gives the policy a structured action representation that may encode properties relevant to Wordle, such as letter identity, vowel structure, repeated letters, or orthographic patterns. We ask whether these pretrained representations provide a useful inductive bias for policy learning and whether their use changes how task-relevant information is organized in the learned policy representations.

To investigate these representations, we follow the general probing approach of [4], who use linear probes to study the acquisition of human-interpretable chess concepts in AlphaZero. Their analysis examines what concepts can be decoded from an agent’s internal representations, when these concepts emerge during training, and where they are represented in the network. We apply the same general idea on a smaller scale to Wordle. In particular, we probe different levels of the learned policies for task-relevant information and compare policies using pretrained LM embeddings with those using random action embeddings.

Our experiments compare a flat RL policy with a hierarchical policy architecture, using either pretrained character-level LM embeddings or fixed random word embeddings as action representations. All agents learn the same Wordle task using Proximal Policy Optimization (PPO). In the hierarchical architecture, the game state is first encoded into a high-level representation and a continuous latent strategy vector, which is then used by the lower-level network to select a word action. We probe the high-level, strategy, and low-level representations to examine where task-relevant information is encoded, and compare LM and random embeddings to assess the contribution of pretrained lexical structure. Through these comparisons, we investigate whether pretrained linguistic action representations facilitate policy learning, whether they affect the organization of task-relevant information within the learned representations, and whether an explicit representational hierarchy gives rise to interpretable internal structure during Wordle solving.

2 Data and Methods

2.1 Wordle Data and Preprocessing

We use the original Wordle word lists extracted from the game’s source code and archived by [5]. The dataset distinguishes between 2,315 possible solution words and 10,657 additional words that are accepted as guesses but are not selected as targets. Their union defines an action space of 12,972 legal five-letter words.

The 2,315 possible targets are randomly divided into training, validation, and test sets using an 80/10/10 split with a fixed random seed of 42. The complete vocabulary remains available as the action space in all conditions, while only the target words are split. The agent can therefore use the same legal guesses during training and evaluation while being evaluated on unseen target words.

The state representation encodes the complete Wordle history up to the current turn. For each position in each previous guess, the guessed letter is represented by a 27-dimensional one-hot vector (26 letters plus padding), while its feedback is represented by a four-dimensional one-hot vector (gray, yellow, green, plus padding). Six possible guesses are encoded, followed by a

seven-dimensional one-hot vector indicating the current turn. This results in a 937-dimensional state representation.

The observation deliberately does not explicitly contain higher-level quantities such as the number of remaining candidate solutions, the number of known letters, or the number of known positions. These quantities must instead be recoverable from the sequence of guesses and feedback and can subsequently be used as targets when probing the learned representations.

After each guess, the candidate solution set is updated by retaining only target words that would produce exactly the observed feedback for that guess. Feedback follows the standard Wordle procedure, with green matches resolved first and yellow matches subsequently assigned according to the remaining occurrences of each letter. This ensures correct treatment of words containing repeated letters.

2.2 Linguistic Action Representations

Although the Wordle action space is discrete, its actions are not independent categories: each action is an English word with systematic character-level structure. We therefore investigate whether pretrained linguistic representations provide a useful inductive bias for learning the Wordle policy.

We use CANINE-S [6], a pretrained tokenization-free encoder that operates directly on character sequences. Before RL training, every legal Wordle word is passed through the frozen encoder. The final hidden states are mean-pooled over non-padding positions to obtain a fixed embedding $\mathbf{e}_w$ for each word w . These embeddings are computed once and remain frozen throughout RL training.

The language model is therefore used only as an action encoder. It does not generate guesses, restrict the vocabulary, or determine whether an action is valid. The legal action space is always defined by the Wordle vocabulary, while the pretrained encoder provides a structured representation of the relationships among these actions.

As a control condition, we generate fixed Gaussian random embeddings with the same shape as the pretrained embeddings. Their global scale is adjusted to approximately match the standard deviation of the CANINE embeddings. These random representations remain frozen in the same way as the LM representations. This allows us to distinguish effects of pretrained linguistic structure from effects that may arise simply from representing actions as continuous vectors.

Together with the two policy architectures described below, this results in a 2×2 experimental design: [Flat, Hierarchical $\times$ LM, Random.]

2.3 Policy Architectures

2.3.1 Flat Policy

The flat policy directly maps the current Wordle state to a representation used for word selection. The 937-dimensional observation $\mathbf{s}_t$ is processed by a two-layer multilayer perceptron (MLP), with 256 hidden units and ReLU activation in each layer:

$$\mathbf{h}_t = f_{\text{flat}}(\mathbf{s}_t), \quad \mathbf{h}_t \in \mathbb{R}^{256}.$$

Rather than learning an independent output parameter for each of the 12,972 actions, the policy scores words using their fixed action embeddings. The state representation and each word embedding are projected into a shared 256-dimensional space:

$$\mathbf{q}_t = W_s \mathbf{h}_t, \quad \mathbf{k}_w = W_w \mathbf{e}_w.$$

The logit associated with word w is computed using a scaled dot product:

$$\ell_t(w) = \frac{\mathbf{q}_t^\top \mathbf{k}_w}{\sqrt{256}}.$$

The projections W_s and W_w are learned during RL training, whereas the underlying LM or random action embeddings remain fixed.

2.3.2 Hierarchical Policy

The hierarchical architecture introduces an intermediate continuous representation between the observed game state and the word-level policy. It is therefore a representational hierarchy rather than an implementation of discrete options or temporally extended actions.

First, the state is processed by a high-level encoder consisting of two 256-dimensional ReLU layers:

$$\mathbf{h}_t^H = f_H(\mathbf{s}_t), \quad \mathbf{h}_t^H \in \mathbb{R}^{256}.$$

A learned linear transformation maps this representation to a 128-dimensional continuous strategy representation:

$$\mathbf{z}_t = W_z \mathbf{h}_t^H, \quad \mathbf{z}_t \in \mathbb{R}^{128}.$$

The strategy representation is then concatenated with the original observation and passed through a low-level encoder:

$$\mathbf{h}_t^L = f_L([\mathbf{s}_t; \mathbf{z}_t]), \quad \mathbf{h}_t^L \in \mathbb{R}^{256}.$$

The resulting low-level representation is used to score word actions with the same candidate-scoring mechanism as in the flat policy.

The architecture therefore exposes three internal representations,

$$\mathbf{h}_t^H \rightarrow \mathbf{z}_t \rightarrow \mathbf{h}_t^L,$$

which can subsequently be examined independently. No semantic interpretation is assigned to the strategy representation during training; instead, we investigate post hoc whether interpretable task information emerges at different levels of the hierarchy.

2.4 Wordle Environment and Reward

Each episode begins by sampling a target from the corresponding target split. The agent is allowed a maximum of six guesses. An episode terminates immediately when the target is guessed correctly and otherwise ends after the sixth unsuccessful guess.

During preliminary experiments, we investigated a shaped reward that explicitly rewarded intermediate information-seeking behavior, including newly discovered yellow letters, newly identified green positions, and reductions in the remaining candidate set. Information gain was defined as

$$I_t = \log |C_t| - \log |C_{t+1}|,$$

where C_t denotes the set of target words consistent with the feedback available before guess t .

These experiments revealed an undesirable interaction between the intermediate objectives and the ultimate objective of solving the puzzle. In particular, rewarding information gain encouraged the policy to continue selecting exploratory guesses even when the candidate set had already become very small and directly attempting a remaining solution would have been

preferable. In other words, the shaped objective could reward further reduction of uncertainty even when additional exploration no longer served the actual task objective.

For the final experiments, we therefore removed the yellow, green, and information-gain components from the training reward. The final reward is

$$r_t = -0.1 + 10S_t,$$

where $S_t = 1$ when the current guess solves the puzzle and $S_t = 0$ otherwise. A non-terminal guess therefore receives -0.1 , while a successful guess receives 9.9 . Intermediate quantities such as information gain and newly discovered letters are still recorded for behavioral analysis but do not contribute to the optimization objective.

This choice is also important for the subsequent interpretability analysis. Properties such as information gain, known letters, and known positions are not themselves explicit reward objectives in the final experiments. Consequently, if such variables become decodable from the learned representations, this cannot be attributed simply to a corresponding reward term.

The candidate set is nevertheless used to provide a soft training-time action bias. Let $m_t(w) \in [0, 1]$ indicate whether word w remains a possible solution according to the feedback observed so far. During training, the action logit is modified according to

$$\ell'_t(w) = \ell_t(w) - \lambda_{\text{mask}}(1 - m_t(w)),$$

where $\lambda_{\text{mask}} = 8$. This does not constitute a hard mask: inconsistent words remain valid actions, but receive a lower logit. The agent can therefore still select exploratory words that are not themselves possible solutions.

2.5 RL Training

All four experimental conditions are trained using Proximal Policy Optimization (PPO). The optimization settings are held constant across policy architectures and action-representation conditions. We use a learning rate of 10^{-4} , discount factor $\gamma = 0.99$, generalized advantage estimation with $\lambda = 0.95$, PPO clipping parameter 0.2 , entropy coefficient 0.01 , and value-function loss coefficient 0.5 .

Each training iteration collects $4,096$ environment steps and performs five optimization epochs with minibatches of 256 samples. Training is parallelized across four environment runners, each containing eight environments, while optimization is performed using one GPU. Each model is trained for 100 PPO iterations.

Validation is performed after every training iteration on the held-out validation targets using deterministic action selection. Model checkpoints are saved every five iterations. Saving intermediate checkpoints allows us to examine not only the final representations but also how task-relevant information develops over the course of training.

The value function of the flat architecture is predicted directly from the 256 -dimensional state representation $\mathbf{h}_t$. In the hierarchical architecture, the value function is predicted from the high-level representation $\mathbf{h}_t^H$. The policy encoders, value functions, and state/action projection layers are optimized by PPO, whereas the pretrained or random word embeddings remain fixed throughout training.

2.6 Representation Probing

We use linear probes to examine what task-relevant information is encoded in the learned representations, following the general approach of [4]. The RL models are frozen during probing.

A fixed probing corpus is generated independently of the learned policies. We use the validation target split defined before RL training to fit and select the probes, and the held-out

test target split for final evaluation. For each target, an independent behavior policy repeatedly samples a guess from the remaining candidate solutions. After each guess, the candidate set is updated according to the observed feedback, and each non-terminal state is added to the probing corpus. Using the same probe states for all models and checkpoints allows direct comparison of their representations.

For the flat policy, we probe the 256-dimensional hidden representation $\mathbf{h}_t$. For the hierarchical policy, we separately probe the high-level representation $\mathbf{h}_t^H$, the 128-dimensional strategy representation $\mathbf{z}_t$, and the low-level representation $\mathbf{h}_t^L$:

$$\mathbf{h}_t^H \rightarrow \mathbf{z}_t \rightarrow \mathbf{h}_t^L.$$

We probe three properties of the current game state: timestep, green count, and the logarithm of the number of remaining candidate solutions,

$$\log |C_t|.$$

Green count and candidate-set size are not explicitly provided as scalar inputs to the policy and must be recovered from the guess–feedback history. We additionally define five binary targets indicating whether each of the five character positions has been identified as green.

For timestep, green count, and log candidate count, we fit ridge regression probes and report test R^2 . Representation dimensions are standardized before fitting, and the regularization parameter is selected from

$$\alpha \in 10^{-3}, 10^{-2}, 10^{-1}, 1, 10, 100, 1000, 10000$$

using five-fold grouped cross-validation on the validation probing set.

For the five position-specific green targets, we fit logistic regression probes with balanced class weights and report balanced accuracy. The regularization parameter is selected from

$$C \in 0.01, 0.1, 1, 10, 100$$

using five-fold stratified grouped cross-validation. States generated from the same target word are kept in the same fold.

After hyperparameter selection, probes are fitted on the validation representations and evaluated on representations generated from the held-out test target split. This procedure is repeated for checkpoints saved every five PPO iterations from iteration 5 to 100.

Following [4], this analysis addresses three questions: *what* task information can be decoded, *where* it is represented in the network, and *when* it becomes decodable during training. Probe performance measures linear decodability and does not imply that the policy causally uses the decoded information.

3 Results

3.1 Training Performance

Figure 1 compares the four combinations of architecture (Flat vs. HRL) and action representation (LM vs. Random). We report the training solve rate and the mean number of guesses among successfully solved episodes.

The strongest difference is between LM and random action representations. Flat+LM improves from a solve rate of approximately 0.45 to 0.86, while HRL+LM improves from approximately 0.48 to 0.90. In contrast, both Random conditions remain around 0.48–0.50 throughout training. Thus, substantial performance is already obtained in the Random conditions under the shared candidate-set bias, but these models show little further improvement during training.

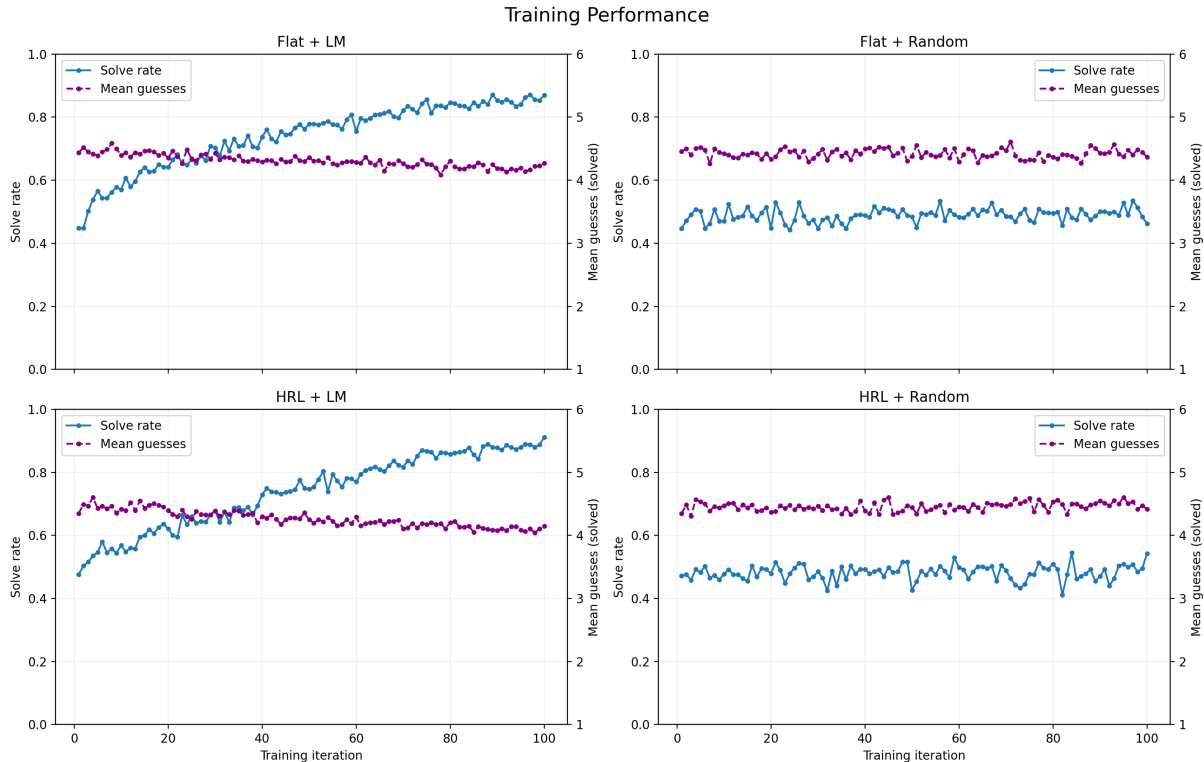


Figure 1: Training performance for the four combinations of architecture and action representation. The blue curves show solve rate, while the purple dashed curves show the mean number of guesses among solved episodes.

The mean number of guesses shows a similar pattern. In the LM conditions, the mean decreases from approximately 4.5 to 4.2 guesses for Flat+LM and to 4.1 for HRL+LM. With random representations, it remains around 4.4–4.5. Both architectures can therefore learn effectively with the pretrained action representations. HRL reaches the highest final solve rate, but the Flat–HRL difference is small compared with the LM–Random difference.

3.2 Probing Learned Representations

Figure 2 shows linear probe performance across training checkpoints. Continuous variables are evaluated using test R^2 , while position-specific green information is evaluated using balanced accuracy. We analyze the results in terms of *what* information can be decoded, *when* it becomes available, and *where* it is represented in the network.

Regarding *what* is represented, all measured task variables are linearly decodable to different degrees. Candidate-set uncertainty, measured by the log number of remaining candidates, reaches approximately $R^2 = 0.7$ at most checkpoints. The number of known green positions shows a similar pattern. Timestep is particularly explicit, reaching approximately $R^2 = 0.98$ –1.0 in the Flat representation and remaining strongly decodable throughout the HRL representations. Position-specific green information is also highly decodable, with balanced accuracy substantially above chance across conditions.

Regarding *when* these representations emerge, most of the measured information is already strongly decodable at the first analyzed checkpoint, iteration 5. Candidate count, green count, timestep, and position-specific green information show relatively little systematic improvement over the remaining training trajectory. Thus, although behavioral performance continues to improve substantially in the LM conditions, linear accessibility of these state properties develops much earlier. A small number of isolated checkpoints, particularly for Flat+Random, show sharp drops in R^2 , but performance immediately recovers at adjacent checkpoints. We therefore treat

these as probe instabilities rather than persistent representational changes.

Regarding *where* the information is represented, the global state variables show little evidence of strict hierarchical separation. Candidate count, green count, and timestep are strongly decodable from the high-level, strategy, and low-level HRL representations, indicating that this information remains distributed across the hierarchy. Position-specific green information shows a clearer organization. The low-level HRL representation consistently achieves the strongest decoding performance, particularly with the LM prior, where mean balanced accuracy reaches approximately 0.88–0.91 and remains above the corresponding high-level and strategy representations across training.

Overall, the hierarchical architecture does not appear to increase the general amount of decodable task information or to separate global state information into distinct levels. Instead, its clearest representational effect is a partial specialization of the low-level representation for fine-grained positional information relevant to word selection. Combined with the training results, these findings suggest different roles for the two experimental factors: pretrained LM action representations primarily support successful policy learning, while the hierarchical architecture primarily affects how task-relevant information is organized within the policy.

4 Conclusion

In this project, we used Wordle as a compact environment to study how linguistic action representations and hierarchical policy structure affect reinforcement learning. We compared flat and hierarchical PPO policies using either pretrained character-level LM embeddings or random embeddings, and analyzed their internal representations using linear probes.

The pretrained LM representations had the clearest effect on learning. Both LM-based agents substantially improved their solve rates during training, whereas the models using random action representations showed little improvement. The difference between flat and hierarchical policies was comparatively small, suggesting that structured action representations are more important for behavioral performance than the hierarchical architecture itself.

The probing analysis further showed that task-relevant information is strongly represented in both architectures. Candidate-set uncertainty, green information, and timestep were already linearly decodable early in training and remained broadly distributed across the hierarchical representations. However, position-specific green information was most strongly represented at the low level of the hierarchical policy, providing evidence for partial functional specialization close to word selection.

In conclusion, our results suggest that pretrained linguistic representations mainly support effective learning over the large discrete action space, while hierarchical structure influences how task information is organized rather than simply increasing how much information is encoded. Wordle therefore provides a simple setting in which both learning behavior and the internal organization of learned representations can be studied in an interpretable way.

5 Discussion

One limitation of the current environment concerns the use of the candidate-set bias. In preliminary experiments without any candidate masking, the large action space of 12,972 words made learning extremely difficult and the policy failed to converge to useful solving behavior. Introducing a candidate mask substantially reduced this search problem and allowed learning to take place. However, a hard mask, which restricts actions entirely to words consistent with previous feedback, made the task considerably easier and removed the possibility of using non-candidate words as informative exploratory guesses. We therefore adopted a soft candidate mask as a compromise, subtracting $\lambda_{\text{mask}} = 8$ from the logits of inconsistent words without

making them unavailable. Although this makes the RL problem tractable while preserving some freedom for exploration, it is still an externally imposed Wordle-specific inductive bias rather than a strategy learned entirely by the agent. Moreover, the choice of $\lambda_{\text{mask}} = 8$ was empirical rather than systematically tuned, and different values change the effective difficulty of the task. A more principled approach would therefore evaluate performance across different mask strengths, including $\lambda_{\text{mask}} = 0$, or develop an alternative mechanism that allows the policy to learn candidate consistency without directly encoding it into action selection.

A second design issue concerns the reward function. Intermediate quantities such as newly discovered green positions, retained greens, yellow letters, and information gain are intuitively relevant to successful Wordle play and were included as reward components in preliminary experiments. However, optimizing these objectives did not always lead to better solving behavior. In particular, information-gain rewards could encourage the agent to continue making exploratory guesses after the candidate set had already become small, rather than committing to a likely solution. Similar problems arise when rewarding individual green or yellow discoveries: these signals measure locally useful progress but are not perfectly aligned with the final objective of solving the word within six guesses. We therefore used only the step penalty and solve reward in the final experiments. This produces a cleaner optimization objective, but also removes potentially informative intermediate learning signals. A better reward design could make these components conditional on the game state, for example by rewarding exploration primarily when uncertainty is high and increasingly favoring exploitation as the candidate set becomes small.

These design choices also affect how the probing results should be interpreted. The strong decodability of candidate uncertainty and green information suggests that the policies preserve task-relevant state structure even though these quantities are not directly rewarded in the final objective. At the same time, candidate consistency influences action selection through the soft mask, so the emergence of candidate-related information cannot be regarded as entirely independent of the environment design. More generally, the results illustrate a trade-off in constructing an interpretable Wordle RL task: stronger inductive biases and shaped rewards can make learning easier, but they also make it harder to determine which strategies and representations emerge from the policy itself. Future experiments that systematically ablate the candidate bias and introduce better-aligned reward shaping would help distinguish learned strategic structure from structure imposed by the environment.

Code Availability

Code for the experiments and analyses in this study is available at https://github.com/WitchPuff/wordle_HRL.

References

- [1] Dimitris Bertsimas and Alex Paskov. An exact solution to wordle. *Operations Research*, 73(3):1384–1394, 2025.
- [2] Siddhant Bhambri, Amrita Bhattacharjee, and Dimitri Bertsekas. Reinforcement learning methods for wordle: A pomdp/adaptive control approach. *arXiv preprint arXiv:2211.10298*, 2022.
- [3] Matthew M. Botvinick, Yael Niv, and Andrew G. Barto. Hierarchically organized behavior and its neural foundations: A reinforcement learning perspective. *Cognition*, 113(3):262–280, 2009.
- [4] Thomas McGrath, Andrei Kapishnikov, Nenad Tomašev, Adam Pearce, Martin Wattenberg, Demis Hassabis, Been Kim, Ulrich Paquet, and Vladimir Kramnik. Acquisition of chess knowledge in AlphaZero. *Proceedings of the National Academy of Sciences*, 119(47):e2206625119, 2022.
- [5] Tyler Glaiel Scholtes. Wordle word lists. <https://gist.github.com/scholtes/94f3c0303ba6a7768b47583aff36654d>, 2022. GitHub Gist, accessed October 2026.

- [6] Jonathan H. Clark, Dan Garrette, Iulia Turc, and John Wieting. CANINE: Pre-training an efficient tokenization-free encoder for language representation. *Transactions of the Association for Computational Linguistics*, 10:73–91, 2022.

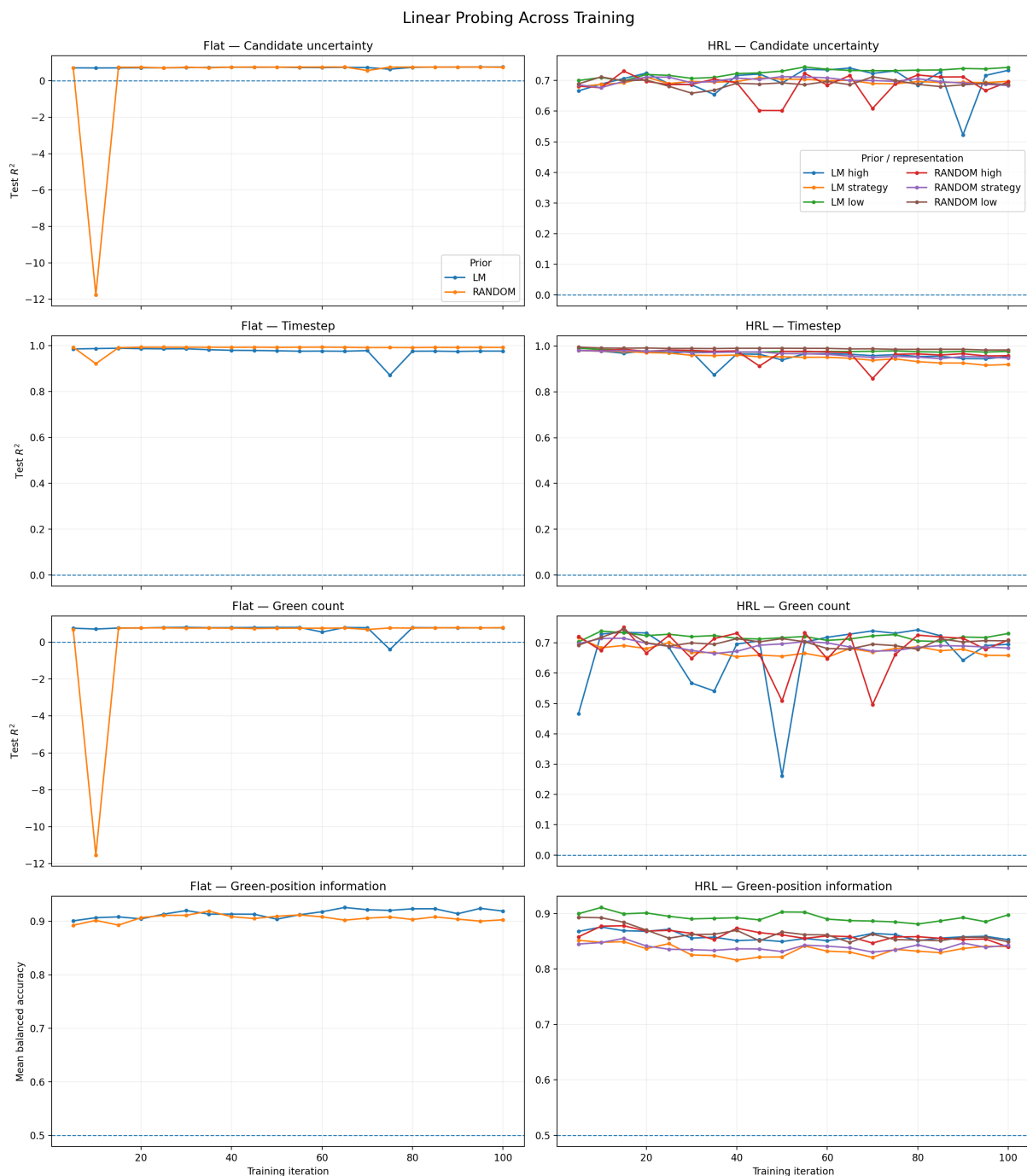


Figure 2: Linear probing results across training. Left panels show the Flat architecture and right panels the HRL architecture. Continuous variables are evaluated using test R^2 ; green-position information is evaluated using mean balanced accuracy across the five character positions. Dashed horizontal lines indicate the corresponding chance or zero-predictive baselines.